

Introduction to nlp & text processing

- Check NLP Session Notebook:
https://colab.research.google.com/drive/1uonOUXUPW98QWoq_a8rzoOEdFcmA4H3?authuser=1

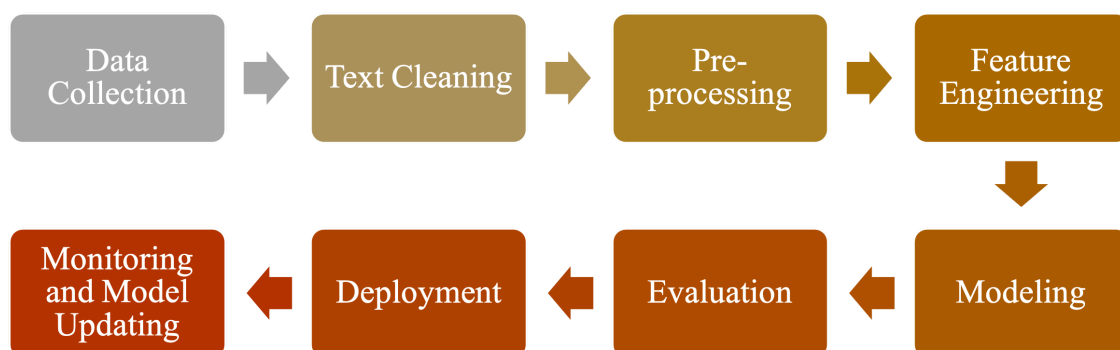
Main AI Areas Nowadays:

1. Classic ML
2. Computer Vision
3. nlp
4. Multi-model = nlp (Text, Audio) + Computer Vision

Image Processing → Image data

Natural Language Processing: Text, Audio Data

NLP Pipeline



Data Collection : Web Scrapping

- Selenium, Helium
- Scrapping Packages : Scrappling, Scrapy, ...
- Octoparse, ParseHub, APIfy ,Scrapping Extensions, ...

NLP Packages:

- spacy, nltk (natural language tool kit)

Data Cleaning :

1. Tokenization: is Tokenization equal to text splitting ? No



```
# tokenization using spacy
```

```
nlp = spacy.load("en")
```

```

doc = nlp("Let's go to the U.K. for $50!")

# Print each token found in the Doc object
tokens = [token.text for token in doc]

print(tokens)
# Output: ["Let", "'s", "go", "to", "the", "U.K.", "for", "$", "50", "!"]

```

```

# tokenization using nltk
from nltk.tokenize import word_tokenize
text = "Hello world! How are you?"
tokens = word_tokenize(text)
print(tokens)

```

- Tokenization in Arabic : Takseem : <https://github.com/ARBML/tkseem> , <https://github.com/riotu-lab/aranizer>
- Arabic leaderboard : <https://huggingface.co/spaces/MohamedRashad/arabic-tokenizers-leaderboard>

2. Sentence Segmentation

Sentence Segmentation

Hello world. This blog post is about sentence segmentation. It is not always easy to determine the end of a sentence. One difficulty of segmentation is periods that do not mark the end of a sentence. An ex. is abbreviations.



- Hello world.
- This blog post is about sentence segmentation.
- It is not always easy to determine the end of a sentence.
- One difficulty of segmentation is periods that do not mark the end of a sentence.
- An ex. is abbreviations.

```
import nltk
nltk.download('punkt')
from nltk.tokenize import sent_tokenize

text = "Hello there! How are you today? I am learning Python."
sentences = sent_tokenize(text)

print(sentences)
```

Arabic Semantic Segmentation : <https://huggingface.co/marioVIC/arabic-semantic-chunking>

3. Stemming & Lemmatization

<https://encrypted-tbn0.gstatic.com/images?q=tbn:ANd9GcR6uG5GA5Ik8tSfkTNGaNE401xXrQZvXc-lc-9p9o5ygECc34Km4HXTWc0O&s=10>

— Stemming

- English Stemmers
 1. Porter Stemmer
 2. Lancaster Stemmer
 3. Snowball Stemmer

```
import nltk
from nltk.stem import PorterStemmer, SnowballStemmer, LancasterStemmer

# Initialize the stemmers

porter = PorterStemmer()
snowball = SnowballStemmer("english")
lancaster = LancasterStemmer()

# Test words
words = ["running", "happily", "flies", "studies", "communication"]

# Apply and print results
for w in words:
    print(f"Word: {w:15} | Porter: {porter.stem(w):10} | Snowball: {snowball.stem(w):10} | Lancaster: {lancaster.stem(w):10}")
```

- Arabic Stemmer : <https://github.com/MaafiHanene/Arabic-Stemmers>
- See Also : Stanza : <https://stanfordnlp.github.io/stanza/>

— Lemmatization:

- In English : Stanza , <https://github.com/bjascob/LemmiInflect>, spacy, nltk

- Spacy

```
import spacy

nlp = spacy.load("en_core_web_sm")

doc = nlp("The children were running faster and studies were being
conducted.")

for token in doc:
    print(f"{token.text:<12} Lemma: {token.lemma_}")
```

- nltk

```
import nltk
from nltk.stem import WordNetLemmatizer

nltk.download('wordnet')
nltk.download('omw-1.4')

lemmatizer = WordNetLemmatizer()

# Examples with context (POS tag)
print(lemmatizer.lemmatize("running", pos="v")) # Output: run
print(lemmatizer.lemmatize("better", pos="a")) # Output: good
print(lemmatizer.lemmatize("cats", pos="n")) # Output: cat
```

- using textblob

```
from textblob import Word

words = ["running", "flies", "better", "children"]

for word in words:
    print(f"{word} → {Word(word).lemmatize()}")
```

- Arabic lemmatization : <https://farasa.qcri.org/lemmatization/>

```

import json
import requests
url = 'https://farasa.qcri.org/webapi/lemmatization/'
text = 'يُشار إلى أن اللغة العربية'
api_key = "#####"
payload = {'text': text, 'api_key': api_key}
data = requests.post(url, data=payload)
result = json.loads(data.text)
print(result)

```

- Arabic lemmatization : <https://github.com/linuxscout/qalsadi>

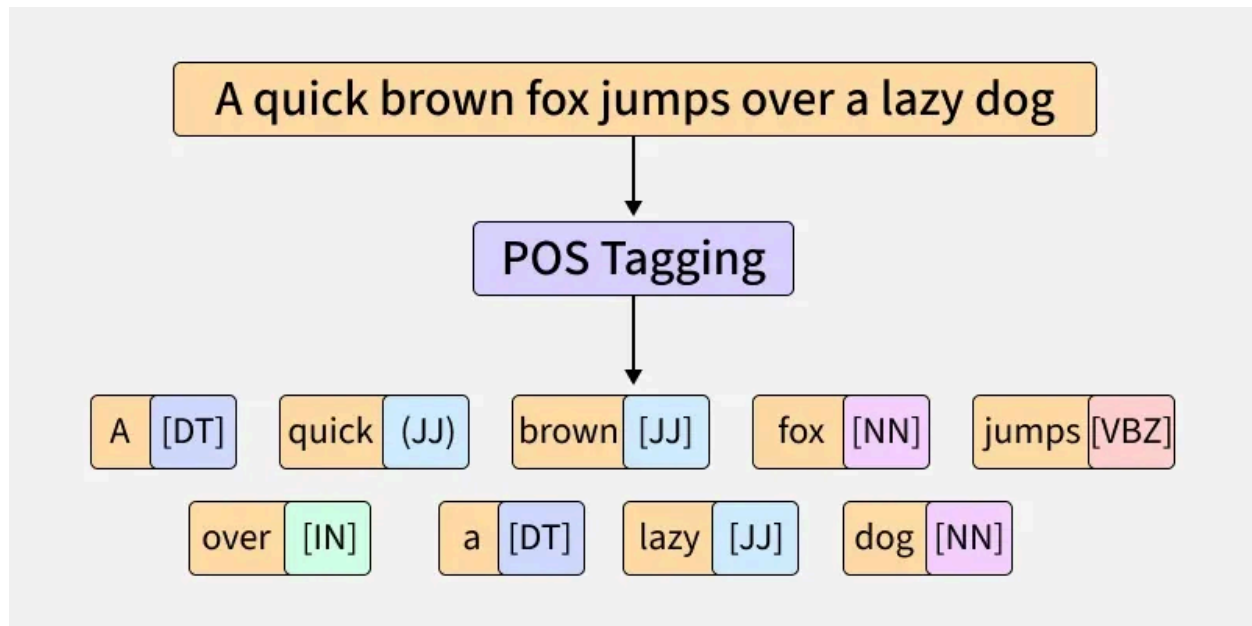
```

>>> import qalsadi.lemmatizer
>>> text = u""""الكلاسيك؟ اللغة "الدارجة" .. الفرنسية
التي ندرس في المدرسة ليست الفرنسية التي يستخدمها الناس في شوارع بار
يس .. وملكة بريطانيا لا تخطب بلغة شوارع لندن .. لكل مقام مقال
"""""
>>> lemmer = qalsadi.lemmatizer.Lemmatizer()
>>> # lemmatize a word
... lemmer.lemmatize("يحتاج")
'احتاج'
>>> # lemmatize a word with a specific pos
>>> lemmer.lemmatize("وفي")
'في'
>>> lemmer.lemmatize("وفي", pos="v")
'وفى'

>>> lemmas = lemmer.lemmatize_text(text)
>>> print(lemmas)
['هل', 'احتاج', 'إلى', 'ترجمة', 'كي', 'تفهم', 'خطاب', 'الملك', '؟', 'ال', 'لغة', 'الكلاسيكي', '(', 'فصحى', ')', 'موجود', 'في', 'كل', 'ال', 'ة', 'ذلك', 'ال', 'ال', 'دارج', '...', 'فرنسي', 'التي', 'درس', 'في', 'مدرسة', 'ليست', 'فرنسي', 'التي', 'استخدم', 'ناس', 'في', 'شوارع', 'باريس', '...', 'ملك', 'بريطانيا', 'لا', 'خطب', 'بلغة', 'شوارع', 'لندن', 'مقال', 'ل', 'و', '...', 'كل', 'مقام', 'مقال']
>>> # lemmatize a text and return lemma pos
... lemmas = lemmer.lemmatize_text(text, return_pos=True)
>>> print(lemmas)

```


4. Part of Speech POS (Tagging)



Tag	Description
CC	Coordinating conjunction
CD	Cardinal number
DT	Determiner
EX	Existential there
FW	Foreign word
IN	Preposition or subordinating conjunction
JJ	Adjective
JJR	Adjective, comparative
JJS	Adjective, superlative
LS	List item marker
MD	Modal
NN	Noun, singular or mass
NNS	Noun, plural
NNP	Proper noun, singular
NNPS	Proper noun, plural
PDT	Predeterminer
POS	Possessive ending
PRP	Personal pronoun

Tag	Description
PRP\$	Possessive pronoun
RB	Adverb
RBR	Adverb, comparative
RBS	Adverb, superlative
RP	Particle
SYM	Symbol
TO	to
UH	Interjection
VB	Verb, base form
VBD	Verb, past tense
VBG	Verb, gerund or present participle
VBN	Verb, past participle
VBP	Verb, non3rd person singular present
VBZ	Verb, 3rd person singular present
WDT	Whdeterminer
WP	Whpronoun
WP\$	Possessive whpronoun
WRB	Whadverb

```
import nltk
from nltk.tokenize import word_tokenize
```

```
# Download necessary datasets first time only
nltk.download('punkt')
nltk.download('averaged_perceptron_tagger')

text = "Python is a powerful programming language."

# NLTK requires text to be tokenized into words first before POS
tokens = word_tokenize(text)
tagged = nltk.pos_tag(tokens)

print(tagged)
```

- Arabic POS : <https://github.com/MohammedAly22/Tasneef>

```
from transformers import pipeline

pos_tagger = pipeline("token-classification", "mohammedaly22/xlm-roberta-base-finetuned-ud-arabic")
text = "اشترى خالد سيارة، و أصبح يمتلك 3 سيارات."

predictions = pos_tagger(text)
words = [item["word"] for item in predictions]
predicted_entities = [item["entity"] for item in predictions]

print(f"words: {words}")
print(f"entites: {predicted_entities}")
```

5. Named Entity Recognition (NER)

https://lh7-us.googleusercontent.com/aLk6DatSTxFNudvqHJ_d5fJNrQmd9CJImHm_iZCE0Oqf1Fq2d-z4n57XHOriCCVcvzA7R4yGg8dgREbl2dsNTr8JcwliRf-QiLaZ5UIBbgushBiGsJRU5v0LqhCikxRLwJ6_p-8PaBskUdCky_qfbxpM

```
import nltk

# Download mandatory components
nltk.download('punkt')
nltk.download('averaged_perceptron_tagger')
nltk.download('maxent_ne_chunker')
nltk.download('words')

text = "Sundar Pichai is the CEO of Google."

# Tokenize, tag POS, and chunk entities
tokens = nltk.word_tokenize(text)
pos_tags = nltk.pos_tag(tokens)
chunks = nltk.ne_chunk(pos_tags)

# Print identified entity subtrees
for chunk in chunks:
    if hasattr(chunk, 'label'):
        entity_name = " ".join(c[0] for c in chunk)
        print(f"Entity: {entity_name:15} | Label: {chunk.label()}")
```

- Arabic NER : <https://github.com/SinaLab/ArabicNER>

6. Stopwords

Common words in a language that are usually filtered out in Natural Language Processing (NLP) tasks because they do not carry significant meaning in text analysis. These words occur frequently in sentences but contribute little to the overall meaning.

Some common stopwords include:

```
"the," "is," "in," "and," "to," "of," "for," "with," "on," "a," "a  
n," "this," "that," "it," "he," "she," "they," "you," "we"
```

- Spacy

```
# print all stopwords  
print(nlp.Defaults.stop_words)  
  
# Check a word is a stopword  
nlp.vocab['word'].is_stop # returns True or False  
  
# Add a word to stopwords  
nlp.Defaults.stop_words.add('btw')  
nlp.vocab['btw'].is_stop = True  
  
# Remove a word from being in stopwords:  
nlp.Defaults.stop_words.remove('beyond')  
nlp.vocab['beyond'].is_stop = False
```

- Arabic Stopwords : <https://pypi.org/project/Arabic-Stopwords/>
- ! pip install Arabic-Stopwords

```
# print arabic stopwords in nltk  
from nltk.corpus import stopwords  
arabic_stopwords = stopwords.words('arabic')  
print(len(arabic_stopwords))
```

7. Translate

- Google Translate :

```
from googletrans import Translator  
  
translator = Translator()
```

```

text = "Hello, how are you today?"
result = translator.translate(text, src='en', dest='ar')

print("Arabic Translation:", result.text)

```

- Translate using Local model : (Argos)

```

!pip install argostranslate

import argostranslate.package
import argostranslate.translate

from_code = "en"
to_code = "ar"

# Download and install Argos Translate package
argostranslate.package.update_package_index()
available_packages = argostranslate.package.get_available_packages()
package_to_install = next(
    filter(
        lambda x: x.from_code == from_code and x.to_code == to_code, available_packages
    )
)
argostranslate.package.install_from_path(package_to_install.download())

# Translate
translatedText = argostranslate.translate("Hello World", from_code, to_code)
print(translatedText)

```

9. Text Summarization

- Summy

```
!pip install sumy
```

```
from sumy.parsers.plaintext import PlaintextParser
from sumy.nlp.tokenizers import Tokenizer
from sumy.summarizers.lsa import LsaSummarizer # You can try Luhn,
LexRank, etc.
```

```
text = """
Artificial Intelligence is transforming industries, from healthcare
to finance.
It enables machines to learn from data, identify patterns, and make
decisions with minimal human intervention.
AI technologies such as natural language processing, computer visio
n, and robotics are increasingly integrated into our daily lives.
"""
```

```
parser = PlaintextParser.from_string(text, Tokenizer("english"))
summarizer = LsaSummarizer()
```

```
summary = summarizer(parser.document, 2) # 2 = number of sentences
in summary
```

```
print("Summary:")
for sentence in summary:
    print("-", sentence)
```